

SQL-rules

2026-08-19

Select Basics

- **SELECT -> FROM -> WHERE**
- **SELECT** attribute-list: choose desired columns/attributes
 - in comma separated list format
 - to select all use *
 - can use expressions
 - can use **DISTINCT** to keep only unique values
 - can use **AS** to create an alias, or temporary name for a column
- **FROM** table-name: specify the table/database where the data is being extracted from
- **WHERE** condition: filters records
 - boolean expression which each row must satisfy
 - can use = to get exact match
 - can use <> not equals operator
 - can use mathematical and string expressions (>, <, etc)
 - can use **AND** for several conditions
 - can use **OR** for either or both (inclusive)
 - can use **XOR** for only either not both (exclusive)
 - can use **NOT** for opposite condition
 - can use **IN** to check if an item is in a list (ie **WHERE** name **IN** (...))
 - can use **LIKE** for pattern matching with % as the wild card (ie **WHERE** name **LIKE** 'D%')
 - can use **BETWEEN** for range checking (ie **WHERE** area **BETWEEN** 200 **AND** 250)

Functions

- **ROUND()**
 - specify value to be rounded, # of decimal places to round to
- **LENGTH()**
 - returns the number of characters in a string
- **LEFT()**
 - isolates the first character
- **CONCAT()**
 - joins two or more strings together into one

Ordering

- By default, SQL sorts data in ascending order.
- **ORDER BY**
 - For descending order: Add **DESC** immediately after the column name (ie **ORDER BY column_name DESC**)
 - **GROUP BY**
 - * Useful to perform aggregate functions on each category
 - * Group rows into buckets, then perform aggregate function
- **HAVING**
 - allows you to filter the groups which are displayed
 - filters after aggregation (**WHERE** filters before aggregation)

Aggregate functions

- performs a calculation on a set of values and returns a single value
- cannot be used in **WHERE**
- if you select an aggregate function alongside an unaggregated column, you must also use a **GROUP BY** clause containing all unaggregated columns
- **COUNT()**
- **SUM()**
- **AVG()**
- **MIN()**
- **MAX()**

Select within Select

- Result of a ***SELECT** may be used as a value in another statement
- Good practice to use **IN** (ie ***SELECT name FROM world WHERE continent IN (SELECT continent FROM world WHERE name='Brazil' OR name='Mexico'))**)
- Operators over a set: **= > < >= <=**
 - binary aka take 2 parameters
 - can use **ANY** or **ALL** for comparison, allows operators to act over a list

Subquery Table, Derived Tables, Nested Queries

- Subquery with FROM: **SELECT** in **FROM**
 - Creates a subtable with selected categories as columns
 - Can calculate new values for the subtable (ie **gdp_per_capita** using **gdp/population**)
 - Calculated values in inner **SELECT** can be used in outer **SELECT**
 - Subtable is given an alias name using **AS**
- Subquery with IN: **SELECT** in ***WHERE***
 - **WHERE** category **IN** (**SELECT** category **FROM** table **WHERE** subcategory = 'name')
 - ie **WHERE** continent **IN** (**SELECT** continent ***FROM** world **WHERE** name = 'Bhutan')
- Correlated Subquery: where value from outer query appears in inner query
 - nested loop
 - finding a stat for a group
 - words like 'each'

Join

- **JOIN**
 - **FROM** -> **JOIN** -> **ON**
 - Combine data
 - **ON** indicates which rows go with the rows in the other dataset
- **INNER JOIN**
 - Only returns rows that have a match in both tables
 - Meaning if one table has age of 7, if the other table does not have an age of 7 that row would not survive after the **JOIN**
- **LEFT JOIN**
 - Keep every single row from the left table (**FROM** __) and bring over matching details from other table if they exist. If there's no match it will put null
- **RIGHT JOIN**
 - * Keep every single row from the right table (**JOIN** __) and bring over matching details from other table if they exist. If there's no match it will put null

Null

- Some datasets have null values for data that is unknown or inappropriate.
- **IS NULL** picks out null fields
- **IS NOT NULL** for fields with a value
- Coalesce
 - **COALESCE**(arg1, arg2, ...)
 - Takes any number of arguments and returns the first value that is not null
 - Returns NULL if all arguments are null
 - Can be useful to replace a NULL value with some other value
- **NULLIF**
 - **NULLIF**(arg1, arg2)
 - returns NULL if the two arguments are equal, otherwise returns the first argument
 - can be used to replace a specific value with NULL
- **NVL**
 - **NVL**(arg1, arg2)
 - exactly like **COALESCE** but can only take 2 args and performs implicit data type conversion on the second arg to match the first (coalesce requires matching types)

Case

- **CASE WHEN** condition **THEN** value
- allows you to return different values under different conditions
- if no conditions match and there is no **ELSE**, **NULL** is returned